

Python E-Course

Module 3 — Pixel Manipulation

Detailed Notes

Module Overview

Level: Foundations

Duration: ~1 week

Prerequisites: Modules 1-2

What You'll Learn

- Read and write individual pixels with the right coordinate order.
- Draw shapes and text on images.
- Extract and modify ROIs (and copy when you must).
- Split and merge channels.
- Apply per-pixel arithmetic with saturation.
- Build masks with bitwise operators.

Lessons

#	Lesson	Duration
1	Accessing Pixels	25 min
2	Drawing Shapes & Text	30 min
3	ROI Extraction & Insertion	25 min
4	Channel Operations	25 min
5	Image Arithmetic	30 min
6	Bitwise Operations & Masking	35 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 4: Color Spaces

Lesson 1: Accessing Pixels

Module: 3 — Pixel Manipulation

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Read and write individual pixels via NumPy indexing.
- Compare `img.item` / `img.itemset` against direct indexing.
- Apply changes to whole rows / columns.

Prerequisites

- Module 1, Module 2

1. Why This Matters

Working pixel-by-pixel is rarely the fastest, but it's the clearest mental model. Once you understand how a single pixel is addressed and modified, every higher-level op (drawing, masking, blending) becomes "do this for many pixels at once."

2. Concept

2.1 Reading

```
img[y, x]          # grayscale: scalar
img[y, x]          # color (BGR): array of 3
img[y, x, 0]       # color: blue channel
```

2.2 Writing

```
img[y, x] = 255          # grayscale to white
img[y, x] = (0, 0, 255)  # color to red (BGR)
img[y, x, 0] = 0         # zero out blue at one pixel
```

2.3 Faster single-pixel access: `item` / `itemset`

```
img.item(y, x)          # ~3x faster than img[y, x] for scalar reads
img.itemset((y, x), 255) # faster write
```

You won't notice the difference for occasional pixels. For tight loops, prefer vectorising entirely (M1 L5).

2.4 Whole rows / columns

```
img[100, :] = 0          # zero out row 100
img[:, 50] = 0           # zero out column 50
img[:, :, 2] = 0         # zero red channel everywhere (BGR)
```

2.5 Per-pixel loops (don't, but know they work)

```
h, w = img.shape[:2]
for y in range(h):
```

```
for x in range(w):
    img[y, x] = 255 - img[y, x]    # negative
```

Slow. The vectorised version is one line: `img = 255 - img`. Save loops for cases where every pixel needs different logic — even then, often `np.where` is cleaner.

3. Code Examples

Example 1: Read & write one pixel

```
import cv2, numpy as np
from skimage.data import coins

img = coins().copy()
print("before:", img[100, 200])
img[100, 200] = 255
print("after :", img[100, 200])
```

Expected Output (console):

```
before: 137
after : 255
```

Example 2: Row / column tricks

```
import cv2, numpy as np
from skimage.data import camera

img = camera().copy()                # (512, 512) grayscale
img[:, 0:5] = 255                    # paint a thin white stripe on the left
img[0:5, :] = 0                      # black stripe on top
cv2.imwrite("camera_stripes.png", img)
```

Expected Output: Cameraman image with a 5-pixel white stripe along the left edge and a 5-pixel black stripe along the top.

Example 3: Whole channel

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR).copy()
img[:, :, 2] = 0                      # zero red channel (BGR index 2)
cv2.imwrite("astro_no_red.png", img)
```

Expected Output: Astronaut with all red removed — image looks cyan-tinted because only blue + green channels remain.

4. Parameter Sensitivity

(No params here — see Module 1 Lesson 5 for whole-image NumPy ops.)

5. Common Mistakes

Mistake	What Happens	Fix
---------	--------------	-----

<code>img[x, y]</code>	Wrong pixel	NumPy is <code>img[y, x]</code>
Looping pixels for trivial ops	Slow	Vectorise: <code>img = 255 - img</code> , etc.
<code>img[y, x] = 256</code> on <code>uint8</code>	Wraps to 0 silently	Clip values to 0..255 first

6. Practice Tasks

- Set `img[50, 50]` to white on a grayscale copy of `coins()`. Verify.
- Zero out row 100 and column 100 on `camera()`. Save and inspect.
- Remove the green channel from `astronaut()`. Save and view.

7. Key Takeaways

- `img[y, x]` reads/writes one pixel. Y first.
- Whole rows/cols/channels via slicing.
- Loops are slow — vectorise when you can.

8. What's Next

Drawing shapes and text on images.

Lesson 2: Drawing Shapes & Text

Module: 3 — Pixel Manipulation

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Draw lines, rectangles, circles, polygons.
- Render text with `cv2.putText`.
- Use `thickness=-1` to fill shapes.
- Use anti-aliasing for smooth edges.

Prerequisites

- Module 3 Lesson 1

1. Why This Matters

You'll annotate detections (bounding boxes), display HUD overlays, build masks programmatically. OpenCV's drawing functions are fast and ubiquitous — and they all use the (x, y) convention.

2. Concept

2.1 The shared signature

Almost every OpenCV draw call follows:

```
cv2.shape(img, geometry, color, thickness=1, lineType=cv2.LINE_8)
```

- `img` is modified in place. Pass a copy if you want to keep the original.
- `color` is (B, G, R) for color images, or a scalar for grayscale.
- `thickness=-1` (or `cv2.FILLED`) fills the shape.
- `lineType=cv2.LINE_AA` enables anti-aliasing.

2.2 Lines

```
cv2.line(img, (x1, y1), (x2, y2), (0, 255, 0), thickness=2)
```

2.3 Rectangles

```
cv2.rectangle(img, (x1, y1), (x2, y2), (0, 0, 255), 2)  
cv2.rectangle(img, (x1, y1), (x2, y2), (0, 0, 255), -1) # filled
```

2.4 Circles

```
cv2.circle(img, (cx, cy), radius=20, color=(255, 0, 0), thickness=2)
```

2.5 Polygons

```
import numpy as np  
pts = np.array([[100, 50], [200, 100], [180, 200], [80, 180]], dtype=np.int32)
```

```
cv2.polylines(img, [pts], isClosed=True, color=(0, 255, 0), thickness=2)
cv2.fillPoly(img, [pts], color=(0, 255, 0))
```

pts must be an array of shape (N, 2) int32 wrapped in a list.

2.6 Text

```
cv2.putText(img, "Hello", (x, y), cv2.FONT_HERSHEY_SIMPLEX,
            fontScale=1.0, color=(255, 255, 255),
            thickness=2, lineType=cv2.LINE_AA)
```

(x, y) is the bottom-left corner of the text baseline — counter-intuitive at first. For nicer fonts, use Pillow (M2 L6).

2.7 Anti-aliasing

lineType=cv2.LINE_AA smooths edges. Slower; use for final output, not video frames.

3. Code Examples

Example 1: Annotate an image with a bounding box and label

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR).copy()

top_left, bottom_right = (180, 60), (340, 220)
cv2.rectangle(img, top_left, bottom_right, (0, 255, 0), 3)
cv2.putText(img, "face", (top_left[0], top_left[1] - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2, cv2.LINE_AA)

cv2.imwrite("astro_annotated.png", img)
```

Expected Output: Astronaut image with a 3-pixel green box around the face region and a "face" label above the top-left corner. Anti-aliased clean edges.

Example 2: Polygon + fill

```
import cv2, numpy as np

canvas = np.zeros((300, 400, 3), dtype=np.uint8)
pts = np.array([[100, 50], [300, 100], [350, 200], [200, 250], [80, 200]],
               dtype=np.int32)
cv2.fillPoly(canvas, [pts], (200, 100, 50))          # filled
cv2.polylines(canvas, [pts], True, (0, 255, 255), 2)  # yellow outline
cv2.imwrite("poly.png", canvas)
```

Expected Output: A pentagon shape filled with brownish-orange, with a bright yellow outline.

Example 3: Multi-shape composition

```
import cv2, numpy as np

canvas = np.full((300, 400, 3), 30, dtype=np.uint8) # dark gray background
cv2.line(canvas, (10, 10), (390, 10), (255, 255, 255), 1)
```

```

cv2.circle(canvas, (200, 150), 60, (0, 255, 0), -1)
cv2.rectangle(canvas, (60, 80), (160, 220), (0, 0, 255), 2)
cv2.putText(canvas, "shapes demo", (50, 280),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 1, cv2.LINE_AA)
cv2.imwrite("shapes.png", canvas)

```

Expected Output: Dark gray canvas with a thin white top border, a filled green circle in the middle, a red unfilled rectangle on the left, and white text "shapes demo" near the bottom.

4. Parameter Sensitivity

Parameter	Low	High
thickness	thin line	fat stroke; -1 = filled
fontScale	small text	huge text
lineType	LINE_8 (fast, jaggy)	LINE_AA (smooth, slower)

5. Common Mistakes

Mistake	What Happens	Fix
Color tuple as (R, G, B)	Colors swapped	Use BGR: (0, 0, 255) is red
Text disappears off-screen	(x, y) was baseline; text drawn upward off the image	Move y down (text baseline is bottom-left)
cv2.polylines(img, pts, ...) instead of [pts]	TypeError	Wrap points in a list
Drawing on original by accident	Original is modified	img.copy() first

6. Practice Tasks

- Draw a yellow circle of radius 50 at the centre of a 300×300 black canvas.
- Add a filled red rectangle around a hard-coded "face" region on astronaut().
- Render "DEMO" in 1.5× scale anti-aliased text at the bottom-right of any image.

7. Key Takeaways

- All drawing functions modify img in place; copy first if needed.
- Coordinates: (x, y) not [row, col].
- thickness=-1 fills.
- lineType=cv2.LINE_AA for smooth edges.

8. What's Next

ROIs — slice patches in and out of images.

Lesson 3: ROI Extraction & Insertion

Module: 3 — Pixel Manipulation

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Extract a rectangular region (ROI) from an image.
- Paste a patch back into a region with matching shape.
- Know when slicing returns a view vs a copy.
- Use ROIs to focus expensive operations.

Prerequisites

- Module 3 Lesson 2

1. Why This Matters

Most useful operations apply to a region, not the whole image — process only the face, blur only the background, replace one logo with another. ROIs make these one-line.

2. Concept

2.1 Extract

```
roi = img[y1:y2, x1:x2]
```

`y1:y2` selects rows, `x1:x2` selects cols. Both ends behave like Python slicing (start inclusive, stop exclusive).

2.2 Insert

```
img[y1:y2, x1:x2] = patch
```

`patch` must have the same shape as the slice. If shapes differ, you get a `ValueError`.

2.3 View vs copy

`roi = img[y1:y2, x1:x2]` is a view — modifying it modifies `img`. Use `.copy()` if you want an independent copy.

```
roi = img[y1:y2, x1:x2].copy()
```

2.4 ROIs across color

For color: `img[y1:y2, x1:x2]` includes all channels. To touch just one channel inside the ROI:

```
img[y1:y2, x1:x2, 0] = 0    # zero blue inside the ROI
```

2.5 Compute then paste

```
roi = img[y1:y2, x1:x2]
roi_blurred = cv2.GaussianBlur(roi, (15, 15), 0)
img[y1:y2, x1:x2] = roi_blurred
```

Blurs only the region. Faster than blurring the full image.

2.6 ROI bounds clipping

OpenCV doesn't crash if you slice past the edge — NumPy gives you the in-bounds part:

```
img[-50:, -50:] # bottom-right 50x50 (whatever exists)
```

But if you compute coordinates from elsewhere (a detection), clip them yourself:

```
y1 = max(0, y1); y2 = min(h, y2)
x1 = max(0, x1); x2 = min(w, x2)
```

3. Code Examples

Example 1: Copy a region to another location

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR).copy()

# Take the face area and paste a duplicate in the top-left corner
face = img[60:220, 180:340].copy() # (160, 160, 3)
img[10:170, 10:170] = face # paste in top-left
cv2.imwrite("astro_dup_face.png", img)
print("face shape:", face.shape)
```

Expected Output: Astronaut image with a 160×160 copy of the face floating in the top-left corner. Console prints face shape: (160, 160, 3).

Example 2: Blur only a region

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR).copy()
y1, y2, x1, x2 = 60, 220, 180, 340

roi = img[y1:y2, x1:x2]
img[y1:y2, x1:x2] = cv2.GaussianBlur(roi, (35, 35), 0)
cv2.imwrite("astro_face_blurred.png", img)
```

Expected Output: Astronaut with the face region heavily blurred (privacy / censor effect), rest of the image sharp.

Example 3: View vs copy demo

```
import numpy as np
img = np.arange(25).reshape(5, 5).astype(np.uint8)
```

```

view = img[1:4, 1:4]
view[:] = 99
print("after view modify:\n", img)

img2 = np.arange(25).reshape(5, 5).astype(np.uint8)
roi = img2[1:4, 1:4].copy()
roi[:] = 0
print("\nafter copy modify (original unchanged):\n", img2)

```

Expected Output:

```

after view modify:
[[ 0  1  2  3  4]
 [ 5 99 99 99  9]
 [10 99 99 99 14]
 [15 99 99 99 19]
 [20 21 22 23 24]]

after copy modify (original unchanged):
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]
 [20 21 22 23 24]]

```

4. Parameter Sensitivity

Slice size	Effect
Tiny (e.g. 5×5)	Local patch ops; cheap
Whole image	No real "ROI" — same as full-image op
Out-of-bounds	Numpy silently clips; manual checks recommended for computed coords

5. Common Mistakes

Mistake	What Happens	Fix
Forgetting .copy()	Modifying ROI also modifies original	Add .copy()
Shape mismatch when pasting	ValueError: could not broadcast ...	Ensure patch shape matches slice
Slicing negative ranges by accident	Empty array	Print roi.shape to confirm
Hardcoded coords past edge	Empty slice instead of error	Clip with max/min

6. Practice Tasks

- Crop a 100×100 ROI from `coffee()` and save it as `coffee_crop.png`.
- Blur only the bottom half of `astronaut()` and save.
- Demonstrate the view-vs-copy difference in a 4×4 array.

7. Key Takeaways

- `ROI = img[y1:y2, x1:x2]`.
- Pasting requires shape match.
- Slices are views — copy if independent.
- Compute-then-paste limits expensive ops to a region.

8. What's Next

Splitting and merging color channels.

Lesson 4: Channel Operations

Module: 3 — Pixel Manipulation

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Split a color image into B, G, R channels.
- Merge channels back into a color image.
- Display single channels as grayscale.
- Swap, zero, or scale individual channels.

Prerequisites

- Module 3 Lesson 3

1. Why This Matters

Many image processing techniques work on one channel at a time — e.g. detect skin in the red channel, find shadows in blue. Splitting and recombining is also the building block for converting between color spaces (Module 4).

2. Concept

2.1 Split

```
b, g, r = cv2.split(img)      # 3 separate (H, W) uint8 arrays
```

Equivalent to:

```
b, g, r = img[..., 0], img[..., 1], img[..., 2]
```

The NumPy form is faster and doesn't allocate intermediate buffers.

2.2 Merge

```
img = cv2.merge([b, g, r])    # back to (H, W, 3)
# or
img = np.stack([b, g, r], axis=2)
```

2.3 Display one channel

A single channel is a 2D grayscale image. Display with `cmap="gray"`:

```
plt.imshow(b, cmap="gray", vmin=0, vmax=255)
```

2.4 Channel swap

```
swapped = cv2.merge([r, g, b])    # swap blue and red
# or
swapped = img[..., ::-1]          # reverse last axis (BGR -> RGB)
```

2.5 Zero a channel

```
no_blue = img.copy()
no_blue[:, :, 0] = 0
```

2.6 Single-channel false-color tint

```
red_only = np.zeros_like(img)
red_only[:, :, 2] = img[:, :, 2]    # keep only the red channel content
```

3. Code Examples

Example 1: Split + display side by side

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
b, g, r = cv2.split(img)

fig, ax = plt.subplots(1, 4, figsize=(14, 4))
ax[0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); ax[0].set_title("color")
ax[1].imshow(b, cmap="gray"); ax[1].set_title("Blue")
ax[2].imshow(g, cmap="gray"); ax[2].set_title("Green")
ax[3].imshow(r, cmap="gray"); ax[3].set_title("Red")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: Four panels. Original color astronaut, then three grayscale views — the blue channel emphasises sky / cool tones; the red channel emphasises skin / helmet; the green is mostly mid-tones.

Example 2: Merge back (round-trip)

```
import cv2, numpy as np
from skimage.data import chelsea

img = cv2.cvtColor(chelsea(), cv2.COLOR_RGB2BGR)
b, g, r = cv2.split(img)
back = cv2.merge([b, g, r])
print("equal?", np.array_equal(img, back))
```

Expected Output: equal? True

Example 3: Swap red and blue (intentionally)

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
swapped = img[..., ::-1].copy()    # reverse last axis
cv2.imwrite("coffee_swapped.png", swapped)
```

Expected Output: Coffee image with channels reversed — red coffee colors now appear blue, browns become teals.

Example 4: Keep only the red channel as a tinted output

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
red_only = np.zeros_like(img)
red_only[:, :, 2] = img[:, :, 2]      # keep red channel
cv2.imwrite("astro_red_only.png", red_only)
```

Expected Output: Astronaut image with only red intensities visible — looks like a red-toned monochrome of the original.

4. Parameter Sensitivity

(No params beyond channel index.)

5. Common Mistakes

Mistake	What Happens	Fix
b, g, r = img (without cv2.split)	Unpacks rows, not channels	Use cv2.split or img[..., i]
Treating channel as RGB on a BGR image	Wrong channel labels	Remember BGR order in OpenCV
Displaying single channel with default cmap	Greenish viridis	cmap="gray"
cv2.merge([r, g, b]) thinking it's RGB output	Still BGR shape, just wrong color	Be explicit about color order

6. Practice Tasks

- Split coffee(), save each channel as a separate grayscale PNG.
- Build a "red-only" version of astronaut() by zeroing green and blue.
- Confirm cv2.merge(cv2.split(img)) is bit-identical to img.

7. Key Takeaways

- cv2.split and cv2.merge reorder data; the cheap NumPy way is img[..., i] and np.stack.
- Each channel is a (H, W) grayscale image.
- Channel zeroing and swapping are one-liners.

8. What's Next

Image arithmetic — adding, subtracting, blending images.

Lesson 5: Image Arithmetic

Module: 3 — Pixel Manipulation

Duration: 30 minutes

Difficulty: Beginner

Learning Objectives

- Add, subtract, and scale images safely (saturation vs wrap).
- Use `cv2.addWeighted` for blending with α / β .
- Build a brightness / contrast effect.

Prerequisites

- Module 3 Lesson 4

1. Why This Matters

Image arithmetic underlies blending, alpha compositing, brightness/contrast, and difference detection. The dtype trap (uint8 wrap-around) bites everyone once — let's get past that.

2. Concept

2.1 NumPy + wraps; cv2.add saturates

```
import numpy as np, cv2
a = np.array([[200]], dtype=np.uint8)
b = np.array([[100]], dtype=np.uint8)

print(a + b)          # [[44]] -> wraps (300 mod 256)
print(cv2.add(a, b))  # [[255]] -> saturates
```

For visual operations on uint8, use `cv2.add` / `cv2.subtract` / `cv2.multiply`.

2.2 Subtract

```
diff = cv2.subtract(a, b)    # 0 if b > a, never negative
```

For signed differences (useful when finding changes), cast to int16 first:

```
diff = a.astype(np.int16) - b.astype(np.int16)
```

2.3 Weighted blend — cv2.addWeighted

```
out = cv2.addWeighted(img1, alpha, img2, beta, gamma)
# out = clip(img1 * alpha + img2 * beta + gamma)
```

For a 50/50 blend:

```
cv2.addWeighted(img1, 0.5, img2, 0.5, 0)
```

Both images must have the same shape and dtype.

2.4 Brightness & contrast

```
# brightness: add a constant
out = cv2.add(img, np.full_like(img, 30))

# contrast: scale around midpoint
out = cv2.convertScaleAbs(img, alpha=1.3, beta=0) # alpha = contrast, beta =
brightness
```

`cv2.convertScaleAbs(img, alpha, beta)` = `clip(alpha * img + beta)` and casts back to `uint8`.

2.5 Difference image (change detection)

```
abs_diff = cv2.absdiff(img1, img2)
```

`absdiff(a, b)` = $|a - b|$, saturated. Useful for motion / change detection.

3. Code Examples

Example 1: Wrap vs saturate

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
wrong = img + 80 # wraps
right = cv2.add(img, np.full_like(img, 80)) # saturates

print("wrong sample px:", wrong[0, 0])
print("right sample px:", right[0, 0])
```

Expected Output (exact pixel values vary):

```
wrong sample px: [22 33 12] <- darker than original due to wrap-around
right sample px: [255 255 92] <- saturated correctly
```

Example 2: Blend two images

```
import cv2
from skimage.data import astronaut, coffee

a = cv2.resize(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR), (400, 400))
b = cv2.resize(cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR), (400, 400))

blend = cv2.addWeighted(a, 0.6, b, 0.4, 0)
cv2.imwrite("blend.png", blend)
```

Expected Output: A 400×400 ghostly overlay of astronaut + coffee, with astronaut more dominant (alpha 0.6 vs 0.4).

Example 3: Brightness & contrast

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
```

```

bright = cv2.convertScaleAbs(img, alpha=1.0, beta=40)    # brightness +40
contrast = cv2.convertScaleAbs(img, alpha=1.5, beta=0)    # contrast 1.5x
both = cv2.convertScaleAbs(img, alpha=1.5, beta=-50)    # both

cv2.imwrite("bright.png", bright)
cv2.imwrite("contrast.png", contrast)
cv2.imwrite("both.png", both)

```

Expected Output: Three saved variants: brighter / more contrast / both. both.png looks punchy with deep shadows.

Example 4: Difference image

```

import cv2
from skimage.data import coffee

img1 = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
img2 = cv2.GaussianBlur(img1, (15, 15), 0)
diff = cv2.absdiff(img1, img2)
print("diff stats: min", diff.min(), "max", diff.max(), "mean", diff.mean())
cv2.imwrite("diff.png", diff)

```

Expected Output: diff stats: min 0 max ~85 mean ~5. The saved diff image is mostly black with brighter speckles where the original had edges (since edges are exactly what blurring loses).

4. Parameter Sensitivity

Param	Low	High
beta (brightness)	dim	bright; clipped to 255
alpha (contrast)	flat / dim	punchy, possibly clipped
alpha blend	second image dominates	first image dominates

5. Common Mistakes

Mistake	What Happens	Fix
a + b on uint8	wrap-around	cv2.add(a, b)
Negative subtract result on uint8	clipped at 0	cast to int16 if you need signed
Different shapes in addWeighted	error	resize / crop first
convertScaleAbs thinking it doesn't clip	it does clip	inspect range with .min()/ .max()

6. Practice Tasks

- Brighten coffee() by 60 using cv2.add. Save.
- Blend astronaut() and coffee() (resize to same shape) at 70/30. Save.
- Compute absdiff between an image and its 15×15 Gaussian-blurred version. Inspect the mean.

7. Key Takeaways

- Use `cv2.add` / `cv2.subtract` for saturating uint8 arithmetic.
- `cv2.addWeighted(a,  $\alpha$ , b,  $\beta$ ,  $\gamma$ )` for blends.
- `cv2.convertScaleAbs(img, alpha, beta)` for brightness/contrast.
- `cv2.absdiff` for change detection.

8. What's Next

Bitwise ops and masking — the workhorse for combining shapes and selections.

Lesson 6: Bitwise Operations & Masking

Module: 3 — Pixel Manipulation

Duration: 35 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Use `cv2.bitwise_and` / `or` / `xor` / `not` for masking.
- Build a mask from a shape or threshold.
- Combine masks; apply a mask to an image.
- Build a basic green-screen effect.

Prerequisites

- Module 3 Lesson 5

1. Why This Matters

A mask is a single-channel binary image (0 or 255) marking "where". Combine masks with images to isolate, replace, or composite regions. This is foundational for segmentation, background removal, and stamp / sticker effects.

2. Concept

2.1 The four bitwise functions

Function	Effect (per pixel)
<code>cv2.bitwise_and(a, b)</code>	bit-AND
<code>cv2.bitwise_or(a, b)</code>	bit-OR
<code>cv2.bitwise_xor(a, b)</code>	bit-XOR
<code>cv2.bitwise_not(a)</code>	invert

For uint8 images, $255 \text{ AND } 255 = 255$, $255 \text{ AND } 0 = 0$, so a mask of 0/255 cleanly selects.

2.2 The masking pattern

```
masked = cv2.bitwise_and(img, img, mask=mask)
```

- mask is a single-channel uint8, same H×W as img.
- Where `mask == 0`, output is 0.
- Where `mask > 0`, output is img.

Equivalent NumPy: `out = np.where(mask[..., None] > 0, img, 0)`.

2.3 Building masks

```
# from a threshold
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)

# from a shape
```

```
mask = np.zeros(img.shape[:2], dtype=np.uint8)
cv2.circle(mask, (200, 200), 80, 255, -1)
```

2.4 Inverting a mask

```
inv = cv2.bitwise_not(mask)
```

2.5 Composite: foreground + background

```
# fg masked by mask, bg masked by inverted mask, then add
fg = cv2.bitwise_and(foreground, foreground, mask=mask)
bg = cv2.bitwise_and(background, background, mask=cv2.bitwise_not(mask))
out = cv2.add(fg, bg)
```

2.6 Color masks (rare but useful)

You can apply bitwise ops between two color images — but for "select region" tasks, build a single-channel mask and use mask= instead.

3. Code Examples

Example 1: Apply a circular mask

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]
mask = np.zeros((h, w), dtype=np.uint8)
cv2.circle(mask, (w // 2, h // 2), min(h, w) // 3, 255, -1)

masked = cv2.bitwise_and(img, img, mask=mask)
cv2.imwrite("astro_circle.png", masked)
```

Expected Output: Astronaut image, but only a circular region in the centre is visible; the rest is solid black.

Example 2: Invert a mask

```
import cv2, numpy as np

mask = np.zeros((200, 200), dtype=np.uint8)
cv2.rectangle(mask, (50, 50), (150, 150), 255, -1)

inv = cv2.bitwise_not(mask)
cv2.imwrite("mask.png", mask)
cv2.imwrite("mask_inv.png", inv)
```

Expected Output: mask.png is a black square with a white inner rectangle; mask_inv.png is the opposite — white square with a black inner rectangle.

Example 3: Composite — green screen replacement

```
import cv2, numpy as np

# Build a "green screen" subject: a synthetic image with green background
subject = np.full((300, 400, 3), (0, 255, 0), dtype=np.uint8) # all green
```

```

cv2.circle(subject, (200, 150), 80, (50, 80, 200), -1)           # tan "face"

# Background: any image of the right size
bg = cv2.resize(cv2.cvtColor(__import__("skimage.data",
fromlist=["coffee"]).coffee(),
                    cv2.COLOR_RGB2BGR), (400, 300))

# Build a mask of "not green" pixels (the subject's foreground)
hsv = cv2.cvtColor(subject, cv2.COLOR_BGR2HSV)
mask_green = cv2.inRange(hsv, (40, 100, 100), (80, 255, 255))  # green range in
HSV
mask_fg = cv2.bitwise_not(mask_green)

fg = cv2.bitwise_and(subject, subject, mask=mask_fg)
bg_only = cv2.bitwise_and(bg, bg, mask=mask_green)
composite = cv2.add(fg, bg_only)

cv2.imwrite("composite.png", composite)

```

Expected Output: The synthetic tan circle floating on the coffee photo (the green background was replaced with the coffee image).

Example 4: Bitwise AND of two shape masks

```

import cv2, numpy as np

a = np.zeros((200, 200), dtype=np.uint8); cv2.circle(a, (80, 100), 70, 255, -1)
b = np.zeros((200, 200), dtype=np.uint8); cv2.circle(b, (120, 100), 70, 255, -1)

both = cv2.bitwise_and(a, b)
either = cv2.bitwise_or(a, b)
xor = cv2.bitwise_xor(a, b)

for name, m in [("a", a), ("b", b), ("and", both), ("or", either), ("xor", xor)]:
    cv2.imwrite(f"{name}.png", m)

```

Expected Output: Five files showing the two overlapping circles and their AND (just the lens of overlap), OR (the full vesica shape), XOR (only the non-overlapping crescents).

4. Parameter Sensitivity

Mask source	Effect
Threshold	binary based on brightness
Shape (cv2.circle etc.)	precise geometric region
HSV color range	object segmentation by color
Edge-based / contour	object outlines (M9-M12)

5. Common Mistakes

Mistake	What Happens	Fix
Mask not single-channel	Operations behave per-channel weirdly	Use (H, W) uint8 mask
Mask non-binary (gray)	Partial selection; soft edges	Use 0 / 255 for hard mask, or

values)	(sometimes desirable)	alpha blending for soft
Mask wrong shape	Crash or misaligned	<code>mask.shape[:2] == img.shape[:2]</code>
Treating <code>cv2.bitwise_and(img, mask)</code> as the masking pattern	Wrong — channels misalign	Use <code>cv2.bitwise_and(img, img, mask=mask)</code>

6. Practice Tasks

- Build a square mask at the centre of `coffee()`; keep only inside the square.
- Build a circular mask and a rectangular mask, AND them, apply to `astronaut()`.
- Implement a tiny green-screen: synthesise a green-background subject, swap the green out for any other image.

7. Key Takeaways

- Mask = (H, W) uint8, 0/255.
- `cv2.bitwise_and(img, img, mask=mask)` is the masking pattern.
- Build masks from thresholds, shapes, or HSV ranges.
- Composite by `cv2.add(fg, bg)` after masking both with mask + inverse.

8. What's Next

End of Module 3. Next: Module 4 — Color Spaces — HSV, LAB, and the color-picking trick for object detection.

Module 3 — Exercises

18 exercises (3 per lesson).

3.1 Accessing Pixels

- (E) Set `coins()[100,200]` to 0 and print before/after.
- (M) Zero out the entire 50th row and 50th column of `camera()`. Save.
- (H) Remove the blue channel from `astronaut()` (BGR index 0).

3.2 Drawing

- (E) Draw a green 5-pixel circle at the centre of a 200×200 black canvas.
- (M) Annotate `astronaut()` with a yellow bounding box and label "face".
- (H) Draw a filled pentagon on a 400×400 canvas using `cv2.fillPoly`.

3.3 ROI

- (E) Crop a 100×100 patch from the centre of `coffee()`. Save it.
- (M) Blur only the bottom half of `astronaut()`. Save.
- (H) Demonstrate view vs copy on a small 5×5 array.

3.4 Channels

- (E) Split `coffee()` into B, G, R; save each as grayscale.
- (M) Confirm `cv2.merge(cv2.split(img))` equals `img`.
- (H) Build a red-only tinted version of `astronaut()`.

3.5 Arithmetic

- (E) Brighten `coffee()` by 60 using `cv2.add`. Save.
- (M) Blend resized `astronaut()` and `coffee()` at 70/30 with `addWeighted`.
- (H) Compute `absdiff` between original `chelsea()` and its 21×21 `GaussianBlur`.

3.6 Bitwise / Masking

- (E) Apply a centered circular mask to `coffee()`.
- (M) Build a mask = circle OR rectangle, apply to `astronaut()`.
- (H) Implement a tiny green-screen: synthesise a green-background subject, swap for `coffee()` as the new background.

Module 3 — Quiz

10 MCQs.

Q1

`img[100, 200]` accesses which pixel? A. row 200, col 100 B. row 100, col 200 C. $x=100, y=200$ D. depends on shape

Answer: B (L1)

Q2

`cv2.rectangle(img, (50, 30), (150, 100), ...)` corners are: A. $(x1,y1)=(50,30), (x2,y2)=(150,100)$ B. $(row,col)=(50,30)$ and $(150,100)$ C. ambiguous D. $(y1,x1)=(50,30)$

Answer: A — OpenCV uses (x, y) . (L2)

Q3

A 100×100 roi = `img[50:150, 80:180]`. Modifying roi modifies: A. nothing B. `img` C. depends D. only if `.copy()`

Answer: B — slice is a view. (L3)

Q4

`cv2.split(img)` for a BGR image returns: A. (R, G, B) B. (B, G, R) C. (gray, alpha) D. (R, G, B, A)

Answer: B (L4)

Q5

`np.array([200], dtype=np.uint8) + np.array([100], dtype=np.uint8)` gives: A. [300] B. [44] C. [255] D. error

Answer: B — wraps. Use `cv2.add` for saturation. (L5)

Q6

`cv2.add` on `uint8`: A. saturates at 255 B. wraps C. errors D. casts to int

Answer: A (L5)

Q7

`cv2.addWeighted(a, 0.5, b, 0.5, 0)` produces: A. equal blend B. only a C. only b D. difference

Answer: A (L5)

Q8

`cv2.absdiff` is used for: A. addition B. absolute difference of two images C. blurring D. masking

Answer: B (L5)

Q9

The masking pattern is: A. `cv2.bitwise_and(img, mask)` B. `cv2.bitwise_and(img, img, mask=mask)`
C. `img * mask` D. `img & mask`

Answer: B (L6)

Q10

For a mask, `cv2.bitwise_not(mask)` does: A. flips horizontally B. inverts the mask C. rotates 90°
D. blurs

Answer: B (L6)

Module 3 — Assignment: Image Watermarking Tool

Estimated time: 2 hours

Combines: Module 3 + Modules 1-2

Brief

Build a watermarking CLI that overlays a text and/or a logo image (with transparency) onto a target image, then saves the result.

Requirements

- `watermark.py INPUT --output OUT [--text "WATERMARK"] [--logo path/to/logo.png] [--opacity 0.5] [--position bottom-right|top-left|center]`
- Text overlay:
- Use `cv2.putText` (or Pillow for nicer fonts).
- Configurable position keyword.
- Add a subtle dark rectangle behind the text for readability.
- Logo overlay:
- Logo should be a PNG with alpha; respect its alpha channel.
- Resize logo so its larger dimension is 1/6 of the target image.
- Blend with `--opacity`.
- Save the result to `--output`.

Constraints

- Module 3 concepts (ROI, arithmetic, bitwise masking).
- Wrap operations in named functions.
- Use `argparse`.

Expected Behaviour (sample)

```
$ python watermark.py astro.png --output out.png --text "(c) Maya" --opacity 0.6 --  
position bottom-right  
saved out.png
```

The output should have "(c) Maya" inside a subtle dark rectangle at the bottom-right, at 60% opacity.

If `--logo` is provided, the logo PNG should appear in the same corner with its alpha channel respected.

Grading Rubric

Criterion	Weight
Text watermark	25%
Logo overlay (alpha)	25%
Opacity control	15%
Position handling	15%

Style + argparse	20%
------------------	-----

Submission

Save as assignment_module3.py.

Module 3 — Mini Project: Green Screen / Background Replacement

Overview

Replace a green background with any other image. Uses HSV masking + bitwise compositing — the classic chroma-key recipe.

Concepts Used

- HSV preview (M4 will go deeper)
- `cv2.inRange` for masking
- `cv2.bitwise_and` / `bitwise_or` (M3 L6)
- ROI sizing (M3 L3)

Features

- `greenscreen.py SUBJECT BACKGROUND [--output OUT]`
- Detect green pixels in HSV, build a mask.
- Replace those pixels with corresponding pixels from `BACKGROUND` (resized to match).
- Save the composite.

Starter Code

`greenscreen.py`:

```
import argparse
import cv2
import numpy as np

def load(path):
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)
    return img

def composite(subject, bg, low=(35, 70, 70), high=(85, 255, 255)):
    bg_resized = cv2.resize(bg, (subject.shape[1], subject.shape[0]))
    hsv = cv2.cvtColor(subject, cv2.COLOR_BGR2HSV)
    green_mask = cv2.inRange(hsv, low, high)
    # green_mask: 255 where green (background), 0 where subject
    fg_mask = cv2.bitwise_not(green_mask)
    fg = cv2.bitwise_and(subject, subject, mask=fg_mask)
    bg_only = cv2.bitwise_and(bg_resized, bg_resized, mask=green_mask)
    return cv2.add(fg, bg_only)

def main():
    p = argparse.ArgumentParser()
    p.add_argument("subject", help="image with green background")
    p.add_argument("background", help="replacement background image")
    p.add_argument("--output", default="composite.png")
    args = p.parse_args()
```

```

subject = load(args.subject)
bg = load(args.background)
out = composite(subject, bg)
cv2.imwrite(args.output, out)
print(f"saved -> {args.output}")

if __name__ == "__main__":
    main()

```

Test Image

If you don't have a green-screen photo, synthesize one:

```

import cv2, numpy as np
img = np.full((400, 600, 3), (0, 255, 0), dtype=np.uint8) # all green
cv2.circle(img, (300, 200), 100, (40, 100, 200), -1)      # tan face
cv2.imwrite("test_subject.png", img)

```

Use datasets/starter/coffee.png as the background.

Expected Output

```

$ python greenscreen.py test_subject.png datasets/starter/coffee.png
saved -> composite.png

```

composite.png shows the tan circle floating on the coffee image — green has been swapped out.

Extension Challenges

- Add --hsv-low and --hsv-high flags so you can chroma-key any color (e.g., blue screen).
- Erode the mask before applying (M10 preview) to remove edge bleed.
- Soft-edge the mask with Gaussian blur on the mask, then use alpha blend instead of hard bitwise.

Grading Rubric

Criterion	Weight
Basic composite works	35%
Background resized OK	15%
Reasonable HSV range	20%
CLI + style	15%
Documentation / README	15%